

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Hafidz Ulum Ramadhani - 5024241014

7 November 2026

1 Pendahuluan

1.1 Latar Belakang

Sistem tertanam (*embedded system*) merupakan sistem komputasi khusus yang dirancang untuk menjalankan fungsi tertentu secara efisien dan menjadi inti dari berbagai perangkat modern, mulai dari peralatan rumah tangga, otomasi industri, perangkat medis, hingga *Internet of Things* (IoT). Seiring meningkatnya kebutuhan akan perangkat yang cerdas dan terhubung, penguasaan sistem tertanam menjadi kompetensi penting bagi mahasiswa teknik komputer. Penguasaan tersebut dimulai dari pemahaman mikrokontroler dan dasar pemrogramannya, sehingga praktikum ini mengenalkan dua platform yang banyak digunakan, yaitu STM32 Blackpill dengan *framework* Arduino serta ESP32 dengan *framework* ESP-IDF dan Arduino. STM32 Blackpill berbasis arsitektur ARM Cortex-M yang umum dipakai di dunia industri, sedangkan ESP32 menawarkan konektivitas Wi-Fi dan *Bluetooth* terintegrasi yang sangat relevan untuk pengembangan IoT. Perbedaan arsitektur dan tingkat abstraksi pada kedua platform tersebut, mulai dari pustaka Arduino yang sederhana hingga ESP-IDF yang lebih dekat dengan perangkat keras, melatih mahasiswa memahami cara kerja mikrokontroler secara lebih mendalam dan fleksibel. Selain itu, antarmuka masukan digital seperti tombol memerlukan konfigurasi yang tepat agar pembacaan logika stabil. Resistor *pull-up* dan *pull-down*, baik eksternal maupun internal, digunakan untuk mencegah kondisi *floating* pada pin GPIO yang dapat menyebabkan pembacaan tidak menentu, sedangkan teknik *debouncing* diperlukan untuk menyaring pantulan sinyal mekanis tombol (*bouncing*) yang dapat terbaca sebagai beberapa penekanan sekaligus. Di sisi lain, sebagian besar besaran fisik di lingkungan, seperti intensitas cahaya, suhu, dan tekanan, bersifat analog sehingga tidak dapat diolah langsung oleh mikrokontroler yang bekerja dengan data digital. Oleh karena itu, pembacaan *Analog to Digital Converter* (ADC) menjadi kemampuan penting untuk mengakses GPIO dalam mode analog, misalnya pada sensor cahaya *Light Dependent Resistor* (LDR) yang nilai resistansinya berubah sesuai intensitas cahaya. Dengan demikian, praktikum ini dilaksanakan untuk membekali mahasiswa dengan pemahaman dan keterampilan dasar dalam mengakses masukan digital dan analog pada mikrokontroler, serta menjadi landasan bagi pengembangan sistem tertanam yang lebih kompleks pada praktikum selanjutnya.

1.2 Dasar Teori

1.2.1 Mikrokontroler STM32 Blackpill dan ESP32

Mikrokontroler adalah sistem komputasi dalam satu chip yang mengintegrasikan prosesor, memori, dan periferal seperti GPIO, ADC, dan antarmuka komunikasi. STM32 Blackpill menggunakan mikrokontroler STM32F411CEU6 berbasis ARM Cortex-M4 dengan frekuensi hingga 100 MHz, memori *flash* 512 KB, dan SRAM 128 KB. Papan ini dapat diprogram menggunakan *framework* Arduino sehingga pemrograman menjadi lebih sederhana. ESP32 merupakan mikrokontroler dengan prosesor dua inti Xtensa LX6 yang berjalan hingga 240 MHz dan memiliki Wi-Fi serta *Bluetooth* terintegrasi. ESP32 dapat diprogram dengan *framework* Arduino yang berorientasi pada kemudahan penggunaan, maupun ESP-IDF (*Espressif IoT Development Framework*), yaitu *framework* resmi dari Espressif yang berbasis sistem operasi waktu nyata FreeRTOS dan memberikan kendali lebih rinci terhadap perangkat keras.

1.2.2 General Purpose Input Output (GPIO)

GPIO adalah pin pada mikrokontroler yang dapat dikonfigurasi sebagai masukan (*input*) maupun keluaran (*output*) digital. Pada mode *input*, pin membaca tegangan dan menafsirkannya sebagai logika *HIGH* atau *LOW* berdasarkan ambang tegangan tertentu. Pin yang dibiarkan tidak terhubung ke sumber tegangan apa pun berada pada kondisi *floating*, yaitu kondisi ketika tegangan pin tidak menentu dan mudah dipengaruhi derau sehingga pembacaan logika menjadi acak.

1.2.3 Resistor Pull-up dan Pull-down

Resistor *pull-up* dan *pull-down* digunakan untuk memberikan keadaan logika default pada pin masukan agar tidak *floating*. Resistor *pull-up* menghubungkan pin ke tegangan suplai (V_{CC}), sehingga pin bernilai *HIGH* saat tombol tidak ditekan dan bernilai *LOW* saat tombol menghubungkan pin ke *ground*. Sebaliknya, resistor *pull-down* menghubungkan pin ke *ground*, sehingga pin bernilai *LOW* saat tombol tidak ditekan dan *HIGH* saat tombol menghubungkan pin ke V_{CC}. Kedua resistor dapat dipasang secara eksternal pada rangkaian atau diaktifkan secara internal melalui konfigurasi mode pin, misalnya `INPUT_PULLUP` pada *framework* Arduino. Resistor internal umumnya bernilai puluhan kiloohm sehingga menyederhanakan rangkaian, tetapi tidak semua pin menyediakannya. Pada ESP32, pin GPIO34 hingga GPIO39 hanya berfungsi sebagai masukan dan tidak memiliki resistor *pull-up* maupun *pull-down* internal.

1.2.4 Debouncing pada Input Tombol

Tombol mekanis tidak menghasilkan perubahan logika yang bersih saat ditekan atau dilepas. Kontak logam di dalam tombol memantul beberapa kali dalam waktu singkat, biasanya beberapa hingga puluhan milidetik, sehingga menghasilkan rangkaian pulsa yang disebut *bouncing*. Mikrokontroler yang bekerja dengan kecepatan tinggi dapat membaca pantulan tersebut sebagai beberapa penekanan terpisah. *Debouncing* adalah teknik untuk menyaring pantulan ini agar satu penekanan hanya terbaca satu kali. Teknik ini dapat dilakukan secara perangkat keras, misalnya dengan filter RC, maupun perangkat lunak, misalnya dengan memberi jeda waktu tunggu setelah perubahan keadaan terdeteksi dan memastikan keadaan tombol tetap stabil sebelum dianggap valid. Pendekatan berbasis waktu menggunakan fungsi seperti `millis()` lebih disarankan dibandingkan `delay()` karena tidak menghentikan jalannya program.

1.2.5 Analog to Digital Converter (ADC)

ADC adalah periferal yang mengubah tegangan analog kontinu menjadi nilai digital diskret yang dapat diolah mikrokontroler. Resolusi ADC dinyatakan dalam jumlah bit, dengan n bit menghasilkan 2^n tingkat nilai. ADC 12 bit, seperti pada STM32F411 dan ESP32, menghasilkan nilai 0 hingga 4095. Hubungan antara tegangan masukan dan nilai digital dinyatakan pada Persamaan 1.

$$V_{in} = \frac{D_{ADC}}{2^n - 1} \times V_{ref} \tag{1}$$

dengan V_{in} adalah tegangan masukan, D_{ADC} adalah nilai hasil pembacaan, n adalah resolusi ADC, dan V_{ref} adalah tegangan referensi, yaitu 3,3 V pada kedua platform. Pin yang digunakan untuk

membaca ADC harus dikonfigurasi dalam mode analog. Pada ESP32, ADC2 tidak dapat digunakan ketika Wi-Fi aktif sehingga pembacaan sensor umumnya memakai pin ADC1.

1.2.6 Sensor Light Dependent Resistor (LDR)

LDR adalah resistor yang nilai resistansinya berubah terhadap intensitas cahaya. Semakin tinggi intensitas cahaya yang mengenai permukaannya, semakin kecil resistansinya, dan sebaliknya. Karena ADC hanya membaca tegangan, perubahan resistansi LDR dikonversi menjadi perubahan tegangan menggunakan rangkaian pembagi tegangan (*voltage divider*) yang disusun bersama resistor tetap. Tegangan keluaran pembagi tegangan ini dibaca oleh pin ADC sehingga intensitas cahaya dapat direpresentasikan sebagai nilai digital.

2 Tugas Pendahuluan

1. **Soal:** Jelaskan perbedaan mendasar antara *framework* Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

   Jawaban: Arduino adalah *framework* dengan tingkat abstraksi tinggi yang menyediakan fungsi sederhana seperti `pinMode()`, `digitalWrite()`, dan `analogRead()`, sedangkan ESP-IDF adalah *framework* resmi Espressif yang berbasis FreeRTOS dan memberikan akses lebih langsung ke perangkat keras. Kelebihan Arduino adalah kurva belajar yang landai, pustaka yang melimpah, dan kode yang portabel antar platform, tetapi kendali terhadap perangkat keras terbatas dan *overhead*-nya lebih besar. Kelebihan ESP-IDF adalah kendali penuh terhadap periferal, dukungan multitasking FreeRTOS, dan optimasi performa serta konsumsi daya yang lebih baik, tetapi pemrogramannya lebih rumit dan hanya untuk keluarga ESP32.

2. **Soal:** Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk di-*flash*, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.

   Jawaban: Pada STM32 Blackpill, pin USB mikrokontroler (PA11 dan PA12) terhubung langsung ke konektor USB tanpa chip USB-to-Serial. Mikrokontrolernya sendiri tidak menyediakan antarmuka serial lewat USB untuk menerima program dari komputer secara langsung, sehingga umumnya dipakai programmer eksternal ST-Link yang memprogram melalui antarmuka SWD. Sebaliknya, papan ESP32 memiliki chip USB-to-Serial (misalnya CP2102 atau CH340) yang mengubah sinyal USB dari komputer menjadi sinyal UART. Sinyal UART ini terhubung ke pin UART0 ESP32, dan *bootloader* bawaan ESP32 menerima program melalui jalur tersebut. Perlu dicatat bahwa STM32F411 sebenarnya memiliki *bootloader* DFU bawaan sehingga dapat di-*flash* lewat USB dengan mengatur pin BOOT0, tetapi ST-Link lebih praktis karena mendukung *flash* otomatis dan *debugging*.

3. **Soal:** Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi *pull-up* dan *pull-down resistor*, baik saat tombol tidak ditekan maupun saat ditekan.

   Jawaban: Pada konfigurasi *pull-up*, resistor menghubungkan pin ke V_{CC} dan tombol menghubungkan pin ke *ground*. Saat tombol tidak ditekan pin bernilai *HIGH*, dan saat ditekan pin terhubung ke *ground* sehingga bernilai *LOW*. Pada konfigurasi *pull-down*, resistor menghubungkan

pin ke *ground* dan tombol menghubungkan pin ke V_{CC}. Saat tombol tidak ditekan pin bernilai *LOW*, dan saat ditekan pin bernilai *HIGH*. Jadi logika kedua konfigurasi saling berkebalikan (*active-low* pada *pull-up* dan *active-high* pada *pull-down*).

4. **Soal:** Jelaskan fenomena *bouncing* pada tombol mekanik, mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.

   Jawaban: *Bouncing* adalah fenomena ketika sinyal tombol berubah-ubah antara *HIGH* dan *LOW* beberapa kali dalam waktu singkat (beberapa hingga puluhan milidetik) sebelum stabil. Hal ini terjadi karena kontak logam pada tombol mekanis bersifat elastis sehingga memantul saat bersentuhan atau terlepas. Mikrokontroler yang cepat dapat membaca pantulan itu sebagai beberapa penekanan. Pendekatan untuk mengatasinya adalah: (a) *debouncing* perangkat keras, misalnya dengan filter RC atau *Schmitt trigger*; (b) *debouncing* perangkat lunak dengan jeda waktu, baik memakai `delay()` maupun pencatatan waktu dengan `millis()` agar program tidak terhenti; dan (c) pembacaan berulang yang memastikan keadaan tombol konsisten selama beberapa kali sebelum dianggap valid.

5. **Soal:** Bagaimana ADC (*Analog-to-Digital Converter*) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.

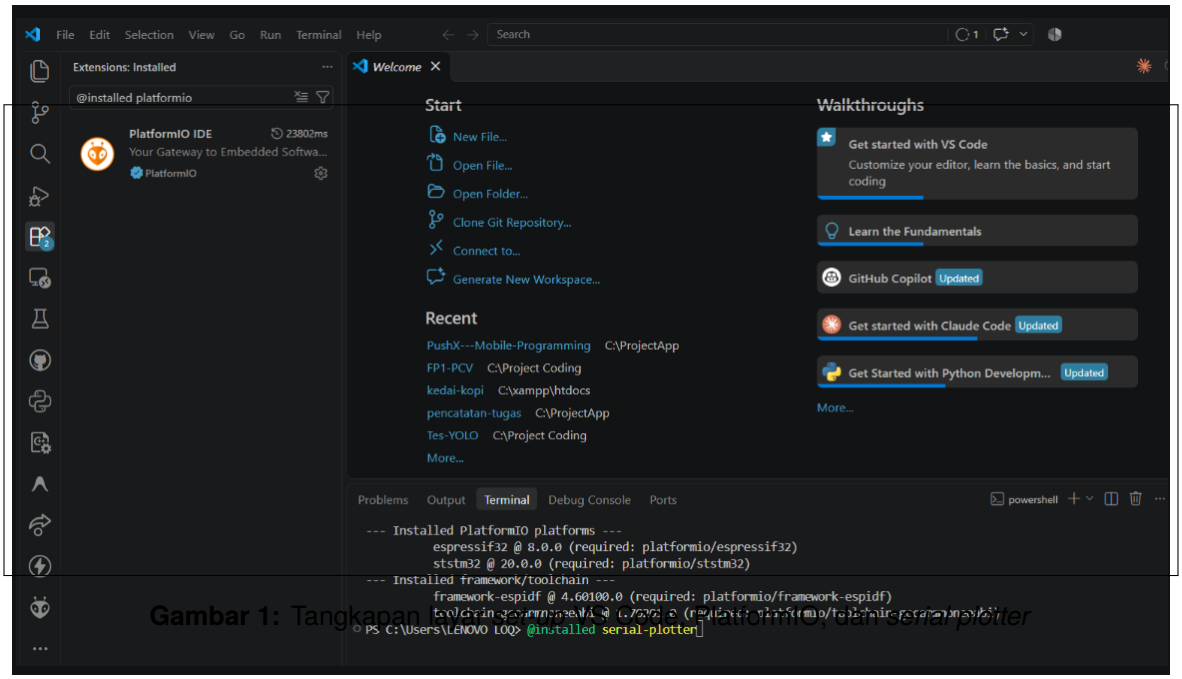
   Jawaban: ADC mengubah tegangan analog menjadi nilai digital melalui proses *sampling* (mengambil cuplikan tegangan pada waktu tertentu), *quantization* (membulatkan tegangan ke salah satu dari tingkat diskret yang tersedia), dan *encoding* (menyatakan tingkat tersebut dalam bilangan biner). Nilai digital yang dihasilkan dinyatakan oleh

$$D_{ADC} = \frac{V_{in}}{V_{ref}} \times (2^n - 1) \tag{2}$$

   dengan n adalah resolusi bit. Semakin besar resolusi, semakin banyak tingkat kuantisasi sehingga semakin kecil tegangan minimum yang dapat dibedakan, yaitu $V_{ref}/2^n$. Sebagai contoh, dengan V_{ref} = 3,3 V, ADC 10 bit memiliki 1024 tingkat dengan resolusi sekitar 3,2 mV, sedangkan ADC 12 bit memiliki 4096 tingkat dengan resolusi sekitar 0,8 mV. Dengan demikian, resolusi yang lebih tinggi menghasilkan pembacaan yang lebih presisi dan galat kuantisasi yang lebih kecil.

6. **Soal:** Lakukan *set-up* untuk *tools*: VS Code, ekstensi PlatformIO (dengan *framework* ESP-IDF dan STM32), dan ekstensi *serial plotter* pada laptop, lalu berikan tangkapan layar bahwa laptop sudah ter-*set-up* dengan baik.

   Jawaban: *Set-up* VS Code, ekstensi PlatformIO (dengan *framework* ESP-IDF dan STM32), serta ekstensi *serial plotter* telah dilakukan pada laptop. Bukti *set-up* ditunjukkan pada Gambar 1.



Gambar 1: Tangkapan layar perintah `@installed` di PlatformIO, dan *serial plotter*